

Lecture 16 — The Producer-Consumer Problem

By Jeff Zarnett and Seyed Majid Zahedi

Classical Synchronization Problems: Producer-Consumer

Various operating systems textbooks provide a few “classical problems”: some scenarios that are phrased in real-world terms but meant to be an analogy for a problem that operating systems will deal with. These standard or classic problems are used to test any newly-proposed synchronization or coordination scheme. The solutions make use of semaphores as the basis for mutual exclusion. We are going to examine three of them in-depth: the producer-consumer problem, the readers-writers problem, and the dining philosophers problem.

The most common synchronization problem is the producer-consumer problem, also sometimes called the bounded-buffer-problem. Two processes share a common buffer that is of fixed size. One process is the producer: it generates data and puts it in the buffer. The other is the consumer: it takes data out of the buffer. This problem can be generalized to have p producers and c consumers, but for the sake of keeping the explanation simple, for now we will have just one of each [Tan08].

There are a couple of rules to be aware of. It is not possible to write into a buffer that is already full; if the buffer has capacity N and there are currently N items in it, the producer cannot write into the buffer and must wait until there is space. It is similarly not possible to read from an empty buffer; if the buffer has zero elements in it, the consumer cannot read from the buffer and must wait until there is something in there.

To keep track of the number of items in the buffer, we will have some variable count. This is a variable shared between more than one thread, and therefore access to this should be controlled with mutual exclusion. Let us assume the maximum number of elements in the buffer is defined as `BUFFER_SIZE`.

If busy-waiting is permitted, that is, we do not care if we are wasting CPU time, we can get away with one mutex, which we can call `mutex`. Each of the producer and consumer threads very likely run in an infinite loop on their own, but the code below is the sufficient to explain one iteration.

Producer

```
1. [produce item]
2. added = false
3. while added is false
4.   lock( mutex )
5.   if count < BUFF_SIZE
6.     [add item to buffer]
7.     count++
8.     added = true
9.   end if
10.  unlock( mutex )
11. end while
```

Consumer

```
1. removed = false
2. while removed is false
3.   lock( mutex )
4.   if count > 0
5.     [remove item from buffer]
6.     count--
7.     removed = true
8.   end if
9.   unlock( mutex )
10. end while
11. [consume item]
```

PC with Semaphores

While this accomplishes what we want, it is inefficient. Let's add a new rule that says we want to avoid busy-waiting. Thus, when the producer is waiting for space it will be blocked and just as the consumer will be when the consumer is waiting for an element. To accomplish this, we will need two general semaphores, each with maximum value of `BUFFER_SIZE`. The first is called `items`: it starts at 0 and represents how many spaces in the

buffer are full. The second is the mirror image spaces; it starts at BUFFER_SIZE and represents the number of spaces in the buffer that are currently empty.

Producer

1. [produce item]
2. wait(spaces)
3. [add item to buffer]
4. post(items)

Consumer

1. wait(items)
2. [remove item from buffer]
3. post(spaces)
4. [consume item]

The producer can continue to produce items until the buffer is full and the consumer can continue to consume items until the buffer is empty. This solution works okay, given two assumptions: (1) that the actions of adding an item to the buffer and removing an item from the buffer add to and remove from the “next” space; and (2) that there is exactly one producer and one consumer in the system. If we have two producers, for example, they might be trying to write into the same space at the same time, and this would be a problem.

To generalize this solution to allow multiple producers and multiple consumers, what we need to do is add another binary semaphore, mutex (initialized to 1), effectively combining the previous solution with the one before it:

Producer

1. [produce item]
2. wait(spaces)
3. wait(mutex)
4. [add item to buffer]
5. post(mutex)
6. post(items)

Consumer

1. wait(items)
2. wait(mutex)
3. [remove item from buffer]
4. post(mutex)
5. post(spaces)
6. [consume item]

This situation should be setting off some alarm bells in your mind. In the synchronization patterns examined earlier, we mentioned the possibility of deadlock: all threads getting stuck. The hint that we might have a problem is one wait statement inside another. Unfortunately, seeing this pattern is not necessarily a guarantee that deadlock is going to happen (that would be too easy). This is, however, a sign that we need to analyze the code to determine if there is a problem.

Reading through the pseudocode above, you should be able to reason that this solution will not get stuck. You may choose a strategy along the lines of “proof by contradiction” and try to come up with a scenario that leads to deadlock. If you are unable to find one, then you may have a suitable solution (though it might be best to have someone else check to be sure). This is not a substitute for a formal mathematical proof, but the logic in your analysis should be convincing. Consider an alternate solution:

Producer

1. [produce item]
2. wait(mutex)
3. wait(spaces)
4. [add item to buffer]
5. post(items)
6. post(mutex)

Consumer

1. wait(mutex)
2. wait(items)
3. [remove item from buffer]
4. post(spaces)
5. post(mutex)
6. [consume item]

This solution is very much like the one we are certain works, except we have swapped the order of the wait statements. As before, we need to analyze this code to determine if there is a problem. This solution does have the deadlock problem. Imagine at the start of execution, when the buffer is empty, the consumer thread runs first. It will wait on mutex, be allowed to proceed, and then will be blocked on items because the buffer is initially empty. The thread is blocked. When the producer thread runs, it waits on mutex and cannot proceed because the consumer thread is in the critical section there. So the producer is blocked and can never produce any items. Thus, we have deadlock. This situation could occur any time the buffer is empty.

If the above pseudocode were implemented it is not a certainty that there will be a deadlock every time. In fact, the code will probably work fine most of the time. Once, however, we have found one scenario that can lead to

deadlock, there is no need to look for other failure cases; we can write off this solution and replace it with a better one.

But let's get to doing an actual example! We will take some time to analyze this solution and understand how we got from the psuedocode above to the actual code below.

Code Example for PC with Semaphore

```
#include <stdlib.h>
#include <pthread.h>
#include <stdio.h>
#include <math.h>
#include <semaphore.h>

#define BUFF_SIZE 100
int buffer[BUFF_SIZE];
int pindex = 0;
int cindex = 0;
sem_t spaces;
sem_t items;
sem_t mutex;

int produce( int id ) {
    int r = rand();
    printf("Producer_%d_produced_%d.\n", id, r);
    return r;
}

void consume( int id, int number ) {
    printf("Consumer_%d_consumed_%d.\n", id, number);
}

void* producer( void* arg ) {
    int* id = (int*) arg;
    for(int i = 0; i < 10000; ++i) {
        int num = produce(*id);
        sem_wait( &spaces );
        sem_wait( &mutex );
        buffer[pindex] = num;
        pindex = (pindex + 1) % BUFF_SIZE;
        sem_post( &mutex );
        sem_post( &items );
    }
    free( arg );
    pthread_exit( NULL );
}

void* consumer( void* arg ) {
    int* id = (int*) arg;
    for(int i = 0; i < 10000; ++i) {
        sem_wait( &items );
        sem_wait( &mutex );
        int num = buffer[cindex];
        buffer[cindex] = -1;
        cindex = (cindex + 1) % BUFF_SIZE;
        sem_post( &mutex );
        sem_post( &spaces );
        consume( *id, num );
    }
    free( id );
    pthread_exit( NULL );
}

int main( int argc, char** argv ) {
    sem_init( &spaces, 0, BUFF_SIZE );
    sem_init( &items, 0, 0 );
    sem_init( &mutex, 0, 1 );
```

```

pthread_t threads[20];

for( int i = 0; i < 10; i++ ) {
    int* id = malloc(sizeof(int));
    *id = i;
    pthread_create(&threads[i], NULL, producer, id);
}
for( int j = 10; j < 20; j++ ) {
    int* jd = malloc(sizeof(int));
    *jd = j-10;
    pthread_create(&threads[j], NULL, consumer, jd);
}
for( int k = 0; k < 20; k++ ){
    pthread_join(threads[k], NULL);
}
sem_destroy( &spaces );
sem_destroy( &items );
sem_destroy( &mutex );
pthread_exit( 0 );
}

```

PC with Monitor

We next look into the implementation of the producer and consumer threads using a monitor. For this purpose, we need a single mutex and two condition variables: *full* and *empty*. *full* will be used for producers to wait when the buffer is full, and *empty* will be used for consumers to wait when the buffer is empty:

Producer

1. [produce item]
2. lock(mutex)
3. while count == BUFF_SIZE
4. cond_wait(full, mutex)
5. end while
6. [add item to buffer]
7. count++
8. cond_signal(empty)
9. unlock(mutex)

Consumer

1. lock(mutex)
2. while count == 0
3. cond_wait(empty, mutex)
4. end while
5. [remove item from buffer]
6. count--
7. cond_signal(full)
8. unlock(mutex)
9. [consume item]

This works fine, but it has a subtle fairness issue, called *starvation*, which happens when a thread never gets a chance to run.

To see how our pseudocode could lead to starvation, remember the ice cream example from the previous lecture (Lecture 14-2). Consider a producer thread A that is waiting on the *full* CV. Suppose that a consumer thread B arrives, consumes an item, and signals *full*. At this point, another producer thread C arrives and goes to the ready queue. The consumer thread B exits. The OS schedules C, which produces an item and makes the buffer full again. The producer thread A runs next and has to wait again. This could continue infinitely many times, starving thread A.

The probability of this happening is extremely low (close to zero!). So, we might be fine with it and leave it as is. However, if we want to make our code 100% starvation-free, our solution will involve a *take-a-number* approach. In this approach, before accessing the buffer, each incoming thread takes a number. They then check the *now-serving* display to see if it's their turn. If not, they wait for their turn.

To implement this solution, we introduce *p_turn* and *c_turn* variables to track turns for producers and consumers, respectively. We also introduce *next_p_turn* and *next_c_turn* variables to indicate the *now-serving* number for producers and consumers, respectively.

Producer

```

1. [produce item]
2. lock( mutex )
3. my_turn = p_turn++
4. while count == BUFF_SIZE ||
   next_p_turn < my_turn
5.   cond_wait( full, mutex )
6. end while
7. [add item to buffer]
8. count++
9. next_p_turn++
10. cond_signal( empty )
11. unlock( mutex )

```

Consumer

```

1. lock( mutex )
2. my_turn = c_turn++
3. while count == 0 ||
   next_c_turn < my_turn
4.   cond_wait( empty, mutex )
5. end while
6. [remove item from buffer]
7. count--
8. next_c_turn++
9. cond_signal( full )
10. unlock( mutex )
11. [consume item]

```

This first attempt is a step in the right direction, but it has a problem. In particular, a signal could be delivered to a ‘wrong’ thread—a thread whose turn has not yet come—who will wake up, check the conditions, and go to sleep again, wasting the signal. When there is equal number of `cond_wait` and `cond_signal`, each wasted signal directly translates to a waiting thread that never wakes up! To solve this issue, we could replace `cond_signal` with `cond_broadcast`:

Producer

```

1. [produce item]
2. lock( mutex )
3. my_turn = p_turn++
4. while count == BUFF_SIZE ||
   next_p_turn < my_turn
5.   cond_wait( full, mutex )
6. end while
7. [add item to buffer]
8. count++
9. next_p_turn++
10. cond_broadcast( empty )
11. unlock( mutex )

```

Consumer

```

1. lock( mutex )
2. my_turn = c_turn++
3. while count == 0 ||
   next_c_turn < my_turn
4.   cond_wait( empty, mutex )
5. end while
6. [remove item from buffer]
7. count--
8. next_c_turn++
9. cond_broadcast( full )
10. unlock( mutex )
11. [consume item]

```

This solves the problem with waking up a ‘wrong’ thread. However, it still has a problem. This problem involves threads that wait because it’s not their turn, not because the buffer is full or empty. Such threads should be signaled by a thread of their own type, because there might be no signals coming for threads of the other type. If the buffer is not empty, a consumer thread leaving the critical section should signal other consumer threads that are waiting. Similarly, if the buffer is not full, a producer thread leaving the critical section should signal other producer threads that are waiting:

Producer

```

1. [produce item]
2. lock( mutex )
3. my_turn = p_turn++
4. while count == BUFF_SIZE ||
   next_p_turn < my_turn
5.   cond_wait( full, mutex )
6. end while
7. [add item to buffer]
8. count++
9. next_p_turn++
10. if count != BUFF_SIZE
11.   cond_broadcast( full )
12. end if
13. cond_broadcast( empty )
14. unlock( mutex )

```

Consumer

```

1. lock( mutex )
2. my_turn = c_turn++
3. while count == 0 ||
   next_c_turn < my_turn
4.   cond_wait( empty, mutex )
5. end while
6. [remove item from buffer]
7. count--
8. next_c_turn++
9. if count != 0
10.   cond_broadcast( empty )
11. end if
12. cond_broadcast( full )
13. unlock( mutex )
14. [consume item]

```

This solution works. However, it is very inefficient in the sense that we wake everyone up, even though only one of them is allowed to proceed.

To properly solve the fairness issue and avoid inefficiency, threads need to send their signal to the ‘right’ waiting thread. We can achieve this by manually creating a FIFO list of waiting threads. Each thread will have their own CV to wait on. Once a thread is done, they signal exactly the next thread in the line:

Producer

```

1. [produce item]
2. lock( mutex )
3. my_turn = p_turn++
4. fifo_push( p_fifo, my_full )
5. while count == BUFF_SIZE ||
   next_p_turn < my_turn
6.   cond_wait( my_full, mutex )
7. end while
8. [add item to buffer]
9. count++
10. next_p_turn++
11. fifo_pop( p_fifo )
12. if count != BUFF_SIZE &&
   !fifo_is_empty( p_fifo )
13.   cond_signal(fifo_head(p_fifo))
14. end if
15. if !fifo_is_empty( c_fifo )
16.   cond_signal(fifo_head(c_fifo))
17. end if
18. unlock( mutex )

```

Consumer

```

1. lock( mutex )
2. my_turn = c_turn++
3. fifo_push( c_fifo, my_empty )
4. while count == 0 ||
   next_c_turn < my_turn
5.   cond_wait( my_empty, mutex )
6. end while
7. [remove item from buffer]
8. count--
9. next_c_turn++
10. fifo_pop( c_fifo )
11. if count != 0 &&
   !fifo_is_empty( c_fifo )
12.   cond_signal(fifo_head(c_fifo))
13. end if
14. if !fifo_is_empty( p_fifo )
15.   cond_signal(fifo_head(p_fifo))
16. end if
17. unlock( mutex )
18. [consume item]

```

Code Example for PC with Monitor

```

#include <stdlib.h>
#include <pthread.h>
#include <stdio.h>
#include <math.h>
#include "fifo.h"

#define BUFF_SIZE 100
#define NUM_PRODUCERS 10
#define NUM_CONSUMERS 10

int buffer[BUFF_SIZE];
int p_index = 0;
int c_index = 0;
int count = 0;

int p_turn = 0;
int c_turn = 0;
int next_p_turn = 0;
int next_c_turn = 0;

pthread_mutex_t mutex;
pthread_cond_t full_cv[NUM_PRODUCERS];
pthread_cond_t empty_cv[NUM_CONSUMERS];

fifo_t p_fifo;
fifo_t c_fifo;

int produce( int id ) {
    int r = rand();
    printf("Producer_%d_produced_%d.\n", id, r);
    return r;
}

void consume( int id, int number ) {
    printf("Consumer_%d_consumed_%d.\n", id, number);
}

```

```

void* producer( void* arg ) {
    int* id = (int*) arg;
    for(int i = 0; i < 10000; i++) {
        int num = produce(*id);
        pthread_mutex_lock(&mutex);
        int my_turn = p_turn++;
        fifo_push(&p_fifo, &full_cv[*id]);
        while(count == BUFF_SIZE || next_p_turn < my_turn) {
            pthread_cond_wait( &full_cv[*id], &mutex );
        }
        buffer[p_index] = num;
        p_index = (p_index + 1) % BUFF_SIZE;
        count++;
        fifo_pop(&p_fifo);
        next_p_turn++;
        pthread_cond_t *c_fifo_head = (pthread_cond_t*) fifo_head(&c_fifo);
        pthread_cond_t *p_fifo_head = (pthread_cond_t*) fifo_head(&p_fifo);
        if(count != BUFF_SIZE && p_fifo_head != NULL) {
            pthread_cond_signal(p_fifo_head);
        }

        if(c_fifo_head != NULL) {
            pthread_cond_signal(c_fifo_head);
        }
        pthread_mutex_unlock( &mutex );
    }
    free( arg );
    pthread_exit( NULL );
}

void* consumer( void* arg ) {
    int* id = (int*) arg;
    for(int i = 0; i < 10000; i++) {
        pthread_mutex_lock( &mutex );
        int my_turn = c_turn++;
        fifo_push(&c_fifo, &empty_cv[*id]);
        while(count == 0 || next_c_turn < my_turn) {
            pthread_cond_wait( &empty_cv[*id], &mutex );
        }
        int num = buffer[c_index];
        buffer[c_index] = -1;
        c_index = (c_index + 1) % BUFF_SIZE;
        count--;
        fifo_pop(&c_fifo);
        next_c_turn++;
        pthread_cond_t *p_fifo_head = (pthread_cond_t*) fifo_head(&p_fifo);
        pthread_cond_t *c_fifo_head = (pthread_cond_t*) fifo_head(&c_fifo);
        if(count != 0 && c_fifo_head != NULL) {
            pthread_cond_signal(c_fifo_head);
        }

        if(p_fifo_head != NULL) {
            pthread_cond_signal(p_fifo_head);
        }
        pthread_mutex_unlock( &mutex );
        consume( *id, num );
    }
    free( id );
    pthread_exit( NULL );
}

int main( int argc, char** argv ) {
    pthread_mutex_init( &mutex, NULL );

    for( int i = 0; i < NUM_PRODUCERS; i++ ) {
        pthread_cond_init ( &full_cv[i], NULL );
    }
    for( int i = 0; i < NUM_CONSUMERS; i++ ) {
        pthread_cond_init ( &empty_cv[i], NULL );
    }
}

```

```

fifo_init(&p_fifo, NUM_PRODUCERS);
fifo_init(&c_fifo, NUM_CONSUMERS);

pthread_t threads[NUM_PRODUCERS + NUM_CONSUMERS];

for( int i = 0; i < NUM_PRODUCERS; i++ ) {
    int* id = malloc(sizeof(int));
    *id = i;
    pthread_create(&threads[i], NULL, producer, id);
}

for( int i = 0; i < NUM_CONSUMERS; i++ ) {
    int* id = malloc(sizeof(int));
    *id = i;
    pthread_create(&threads[i + NUM_PRODUCERS], NULL, consumer, id);
}

for( int i = 0; i < NUM_PRODUCERS + NUM_CONSUMERS; i++ ){
    pthread_join(threads[i], NULL);
}

pthread_mutex_destroy( &mutex );

for( int i = 0; i < NUM_CONSUMERS; i++ ) {
    pthread_cond_destroy(&full_cv[i]);
}

for( int i = 0; i < NUM_CONSUMERS; i++ ) {
    pthread_cond_destroy(&empty_cv[i]);
}

fifo_destroy(&p_fifo);
fifo_destroy(&c_fifo);

pthread_exit(0);
}

```

References

[Tan08] Andrew S. Tanenbaum. *Modern Operating Systems, 3rd Edition*. Prentice Hall, 2008.